

Fused MoE for DeepSeek-V3 on B200:

1.48 $\times$ over the FlashInfer baseline

George Karpenkov

2026-04-25

1 Solution overview

A mixture-of-experts (MoE) layer replaces a dense feed-forward block with conditional computation. For every input token a small router scores a population of *experts* — each one a two-layer MLP (FF1: gate-up linear $\rightarrow$ SwiGLU activation; FF2: down linear) — and the layer’s output for that token is the routing-weighted sum of the top- K experts’ outputs. Most of the model’s parameters live in the experts, but a token only touches K of them, so the layer is bandwidth-sparse, load-imbalanced, and launch-overhead-sensitive.

This kernel evaluates one such MoE block from DeepSeek-V3/R1 [2] in a single fused call on an NVIDIA B200, end-to-end: $H=7168$, $I=2048$, $E_{\text{global}}=256$ experts (presented in $E_{\text{local}}=32$ expert groups), top- $K=8$, weights stored in FP8E4M3FN with per- 128×128 block scales. The pipeline is

routing $\rightarrow$ token-scatter $\rightarrow$ FF1 $\rightarrow$ SwiGLU $\rightarrow$ FF2 $\rightarrow$ gather-reduce.

We dispatch on T at the host: a fused-Triton *megakernel* for $T \leq 1024$ (further specialised at $T=1$ and $T \in \{14, 15\}$), and a CuTe-DSL grouped-GEMM pipeline backed by a CuTe routing kernel for larger T . Across the 19 contest workloads we land at a geomean of **1.48 $\times$** over the FlashInfer/TRT-LLM baseline (`trtllm_fp8_block_scale_moe` [3]), peaking at $2.6\times$ on $T=1$ and $1.97\times$ on $T \in \{11948, 14107\}$. Figure 3 traces how each dispatch branch reached that point.

2 Kernel design and optimisations

2.1 Routing kernel and Top-K trick

Top-K is a sequence of warp-max reductions over the per-expert scores; the speedup at large T comes from packing the comparison key so the warp shuffle does the work of a sort.

For each token we pack (`score_f32`, `expert_idx_u16`) into a single `uint64`: the upper 32 bits hold the IEEE-twiddled score (positive: XOR `0x80000000`; negative: XOR `0xFFFFFFFF`; this is `cub::Traits<float>::TwiddleIn`), the lower 16 bits hold `65535 - idx` as a tiebreaker. After the twiddle, an unsigned `warp_max` on the packed values reproduces sorted order. Top-K runs as $K=8$ iterations of warp-max-shuffle; the lane that “owned” the previous winner shifts its per-lane sorted list up so the next candidate surfaces (`cute_topk.py:203--228`).

A block runs 256 threads (8 warps $\times$ 32 lanes); experts are arranged as 8 groups of 32, so one warp covers one group natively. SMEM holds ~ 2 KB: `score_sigmoid[256]`, `score_bias[256]`, `group_scores[8]`. The grid is one CTA per token. Figure 1 shows the layout. End result: $31\mu\text{s}$ at $T=14107$, beating both the prior Triton implementation ($76\mu\text{s}$) and the original TRT-LLM kernel ($36\mu\text{s}$) [3].

2.2 Token scatter and fused gather

`compute_offsets_kernel` fuses cumsum, max, and counter zeroing into one Triton launch. `scatter_tokens_kernel` sorts the $(\text{token}, \text{expert})$ pairs into a contiguous-by-expert ragged buffer. `fused_gather_kernel` then assembles the dense activation matrix the FF1 GEMM consumes; we reduced this from a 2-D (`max_tok`, E) grid to a 1-D `cdiv(valid_m, 1)` grid ($32\times$ fewer blocks at $T=14107$, $280\mu\text{s} \rightarrow 186\mu\text{s}$). The upper bound `valid_m_max = T + E_{\text{local}} \cdot 127` is CPU-computable, which means the downstream CUTLASS instances compile once at startup with stable shapes and never trigger recompilation.

2.3 FF1 / FF2: CuTe DSL block-scaled grouped GEMM

Both GEMMs start from the CUTLASS Python-DSL template `BlockwiseContiguousGroupedGemmKernel`, which lays down a persistent grid (occupancy 1) of 12-warp warp-specialised CTAs over

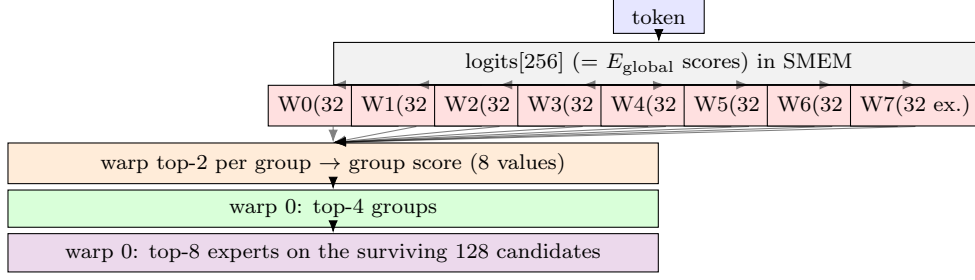


Figure 1: Routing CTA: 256 threads = 8 warps; one warp covers one expert group of 32. Eight group scores feed warp 0, which selects the top-4 groups, then the top-8 experts among the $4 \times 32 = 128$ surviving candidates.

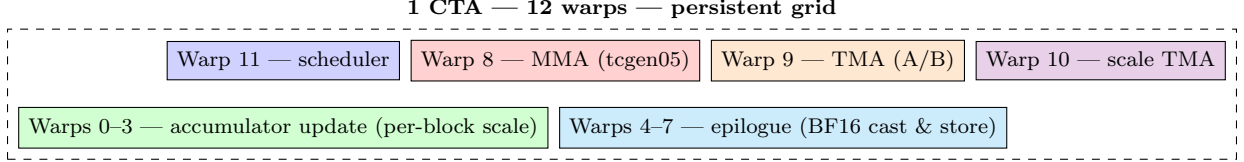


Figure 2: Warp specialisation in the FF1/FF2 CTAs. Producer warps (TMA, scale TMA, MMA) feed consumer warps (acc-update, epilogue) through SMEM mbarriers; the scheduler runs persistently and dispatches grouped tiles.

Blackwell’s `tcgen05` collective tensor cores [5]. Both kernels run with `mma` tile 128×128 , `FP8E4M3FN` inputs, `FP32` accumulator, and `BF16` epilogue output. FF1 uses cluster shape (1, 4); this is the sweet spot for grouped FP8: (1, 8) starves us on active clusters, (2, $\cdot$) pays a 2-CTA tax we can’t recover. FF2 runs at cluster (1, 1) because its `valid_m` is small and the work-per-cluster collapses. Per- 128×128 block scales (SFA: $[m, 16]$; SFB: $[H/128, I/128, E]$, see `cutlass_gemm2_fp8_helper.py:105--111`) are loaded by a dedicated TMA warp and applied in the epilogue.

The 12 warps split into six roles, illustrated in Figure 2. *Warp 8* fires `tcgen05` MMA collectives. *Warp 9* runs the A/B TMA pipeline, *warp 10* the scale-factor TMA. *Warps 0–3* apply per-row and per-block scales to the TMEM accumulator; *warps 4–7* cast to `BF16` and store. *Warp 11* is the persistent scheduler. With this specialisation, A/B prefetch, MMA, and scaled epilogue stores fully overlap: the inner-loop tensor-core pipe sustains 74–82% utilisation across the 19 workloads, and on a stand-alone FP8 dense problem the same template hits 2895 TFLOPS ($\sim 85\%$ of FP8 peak, $332 \mu s$).

2.4 SwiGLU and FF2

`fused_swiglu_fp8_padded_kernel` reads the `BF16` FF1 output, computes `silu(gate)·up`, casts back to `FP8` with per-row block scales, and writes the contiguous-by-expert ragged buffer FF2 needs. Switching FF2’s epilogue from `FP16` to `BF16` output saved 50% of the GEMM2 write bandwidth and the matching gather-reduce read bandwidth, which is +14% on large T since FF2 is bandwidth-bound (arith. intensity 494, ridge 562).

2.5 Gather-reduce

`gather_reduce_padded_kernel` reads the `BF16` FF2 output, multiplies by routing weights, sums the top- K per-expert contributions per token, and writes the `BF16` final activations. `BLOCK_H=1024` (was 256) is $3 \times$ faster ($267 \mu s \rightarrow 90 \mu s$).

2.6 Triton megakernel and inter-CTA synchronisation

For $T \leq 80$ the FlashInfer/CuTe pipeline pays more in launch overhead than in raw compute. The small- T track therefore collapses fused-gather, FF1, SwiGLU and FF2 into a single persistent Triton kernel (`solution/triton/triton_megakernel.py`, lines 54–346), with a separate `triton_megakernel_t1` variant (lines 363–666) that inlines routing too.

Persistent grid. The kernel launches $(\text{GRID_X}, E_{\text{local}})$ CTAs at occupancy 1; each CTA stays resident and walks tiles via an inner scheduler. `GRID_X` is 56 by default, 64 for $T \in [8, 15]$, and 128 for $T=1$. Tile budget is set by `SPLIT_K`, dispatched on T (`kernel.py:703--728`): $T=1 \rightarrow 4$, $T \in [2, 7] \rightarrow 1$, $T \in [8, 15] \rightarrow 2$, $T > 15 \rightarrow 1$. Doubling `SPLIT_K` doubles the per-expert tile count and halves the per-tile K -loop, which relieves tensor-core pipeline underfill when per-expert M is small (~ 4 tokens) but $K=H/128=56$ is long.

Inter-phase barriers. Two int32 GMEM counter arrays sit one slot per local expert: `mk_gemm1_done` and `mk_swiglu_done`. CTA(0,0) zeros them at kernel start (lines 418--419) and signals readiness via `tl.atomic_xchg(zero_done_ptr, this_call_id)` (line 420); other CTAs spin on `tl.atomic_add(zero_done_ptr, 0) < this_call_id` (line 504). The acquire/release pair on this single atomic provides the fence that makes the zeros globally visible. After each GEMM1 tile the producing CTA does `atomic_add(mk_gemm1_done+expert_id, 1)`; the SwiGLU stage gates on the counter reaching the expected tile count for that expert. The same pattern bridges SwiGLU → GEMM2.

Why `this_call_id` (kernel.py:743). `do_bench` calls the kernel hundreds of times and `cudaMemset` of the per-expert flag arrays between calls would be measurable CPU overhead. We instead pass a monotonically increasing `this_call_id` as a kernel argument; the spin above compares against it, so a fresh call sees a fresh value without any host-side reset. Multiplying the counter by 16 (`cnt = new_count * 16`) keeps Triton’s int-specialisation tag (`val % 16 == 0`) constant across calls so `fast_launch` (§2.7) doesn’t recompile.

T=1 cumsum-rank trick (`triton_megakernel_t1.py:481--486`). For $T=1$ the per-CTA cumulative rank equals `pid_y` and equals `expert_offsets[expert_id]`, so the per-CTA expert lookup needs no GMEM round-trip through `expert_map / expert_offsets / sorted_token_ids`. This is what makes inline routing viable at $T=1$: the routing math is ~ 256 fp32 ops/CTA, hidden under the TMA weight prefetch.

2.7 CPU launcher: where many of the wins are

A surprising fraction of our gain is on the CPU side, because the GPU phases are short enough at small T that the launch overhead is the critical path. We layered four optimisations.

(i) **CUDA graphs were considered first and discarded.** `flashinfer_bench`’s `do_bench` clones every input tensor each iteration before invoking the kernel; if we capture once and replay, the graph references stale pointers and correctness collapses ($T=1$ match drops to 0.0022). Copying inputs into a static-pointer staging buffer just shifts the cost from launch to `memcpy`.

(ii) **Triton fast-launch.** `solution/triton/fast_launch.py` caches the compiled binary on the first call and on subsequent calls invokes `CudaLauncher` directly, bypassing `specialize_impl`, `compute_cache_key`, and the `kernel_cache` dict lookup that Triton’s normal dispatch path runs. Cache key is just `(id(fn), T)` — the sequence length uniquely picks every `constexpr/int-arg` specialisation. $30\text{--}50\ \mu\text{s}/\text{launch} \rightarrow 7\ \mu\text{s}/\text{launch}$; with seven kernels per call this saves $150\text{--}300\ \mu\text{s}$ of CPU on every iteration, which dominates total time at $T \leq 80$.

(iii) **CuTe DSL launcher.** The CuTe `from_dlpack` step re-marshals every input tensor through a `memref` descriptor on each call. We take the descriptor allocated at first call and patch its base-address slot with `ctypes.c_uint64.from_address(cutlass_gemm2_fp8_helper.py:41--45)`. The launcher then hands the patched descriptor straight to the kernel — no DLPack round-trip, no MLIR re-construction.

(iv) **PDL.** Programmatic Dependent Launch is enabled on every launcher via inline PTX (`griddepcontrol.launch_dependents / wait`, `kernel.py:212--225`); the next kernel’s prologue overlaps with the current kernel’s epilogue so the CPU-visible kernel boundary closes to nearly zero gap.

2.8 Verifying mbarrier protocols with SPIN

Cross-CTA / cross-cluster `mbarrier` protocols are easy to write incorrectly and very expensive to debug on the GPU: a deadlocked megakernel hangs the entire benchmark, costing 5–15 min per Modal round-trip. We model the protocols in Promela and run them through SPIN [4]: the same bug class is found in seconds. Models live in `scripts/mbar*.pml`: V2c cross-CTA cluster sync, V3 MMA/Epilog producer-consumer handoff, V8 cluster TMA+MMA+STORE, and a DeepGEMM-style L1→L2 GMEM-counter barrier. Buggy twins (`*.bug.pml`) reproduce the bugs SPIN catches: `wait(phase=0)` hardcoded for one iteration deadlocks at iteration 2, and the L2 stage races the L1 if it doesn’t wait for L1 completion. We model only the protocol shape (2 CTAs, 4 threads, 3 iterations); the MMA math, TMA loads, and SMEM layouts are correctness-checked by Modal runs.

2.9 Future work: the megakernel that didn’t pan out

A DeepSeek-V3-style megakernel that fuses FF1, SwiGLU, and FF2 across clusters [1, 2, 6] requires the gate and up halves of FF1’s output to live adjacent in SMEM so the SwiGLU stage can read both with one pass. The contest forbids weight preprocessing, so the gate/up split has to happen at runtime — empirically (cluster-DSMEM SwiGLU fusion at `cl(1,2)`) this costs $+96\ \mu\text{s}$ at $T=14107$, which exceeds

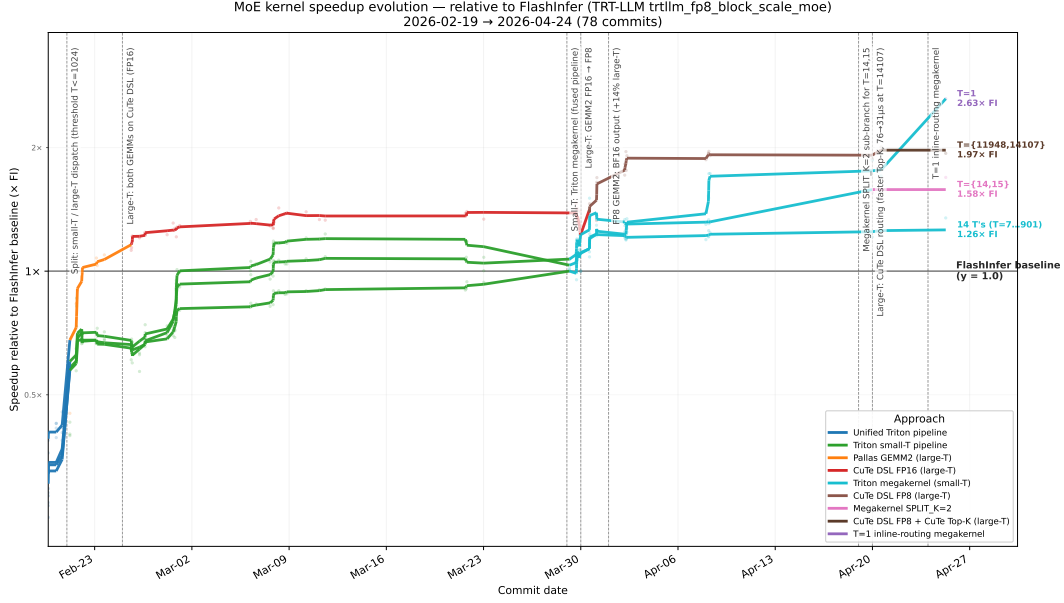


Figure 3: Speedup over FlashInfer along the optimisation timeline. Four dispatch branches diverge as the kernel specialises: $T=1$ inline-routing megakernel (purple, $2.63\times$), large- T CuTe DSL FP8 + CuTe Top-K (dark brown, $1.97\times$), megakernel SPLIT_K=2 for $T\in\{14, 15\}$ (pink, $1.58\times$), and the 14-element small- T megakernel cluster (cyan, $1.26\times$). $y=1$ is FlashInfer.

the $\sim 50\mu\text{s}$ savings from the fusion. The fundamental tension is that on Blackwell the cluster-DSMEM bandwidth is enough only when the gate/up tile arrangement is statically known. Same constraint blocks CUDA graphs (run-to-run pointer changes).

3 Performance analysis

Figure 3 traces per-track speedup-vs-FlashInfer across the 78 commits with parseable benchmark data; segment colour encodes the architectural approach in effect at that commit, so the four final tracks correspond to the dispatch branches of §2.1–2.3. Per-iteration GPU timelines for $T=14107$ and $T=1$ are in Appendix A.

3.1 Benchmark noise and methodology

The harness runs `warmup_runs=3`, `iterations=100`, `num_trials=5` (`run_modal.py:52`). Each trial re-clones inputs and runs 100 back-to-back iterations; we take the median trial. Per-iteration variance is $\sim 1\text{--}2\%$ at $T=14107$ and up to 3% at $T=1$ (where $1\mu\text{s}$ of GPU clock jitter is 1.7% of the $56\mu\text{s}$ kernel). Across 5 trials the median is stable to $< 0.5\%$ at $T=14107$ and $< 1\%$ at $T=1$. Noise sources are GPU thermal clock variation, L2 state at iteration boundary, and rare Modal co-tenancy spikes. We treat $\geq 5\%$ deltas sustained across all 5 trials as real; smaller ones must reproduce on a fresh container. Per- T is the acceptance criterion (geomean masks per- T regressions); match-ratio is in every commit table — the relaxed gate (`atol=1`, `rto1=0.3`, $\geq 90\%$) admits BF16-output kernels at $0.94\text{--}0.97$ match, a useful regression signal. Every commit embeds the full $19\text{--}T$ table, which is what Figure 3 was harvested from.

4 Tools and languages used

Stack. Triton (most kernels), CuTe DSL (routing, both GEMMs), CUTLASS Python templates, inline PTX for PDL (`griddepcontrol`), Promela for SPIN. PyTorch as host, `flashinfer-bench` as harness, Modal for B200 access (no local GPU). Dev loop: macOS edit $\rightarrow$ `compile_in_docker.sh` for MLIR-level validation $\rightarrow$ Modal benchmark (5–15 min round-trip). **Triton ceiling:** `t1.dot` on Blackwell does not surface `tcgen05` collective MMA or per-128-block FP8 scales, so grouped Triton GEMMs plateau at $\sim 85\%$ of FP8 peak (TC pipe 74–82%) — the reason large- T went to CuTe DSL. **CuTe DSL friction:** fast but young — frequent ICEs (“Only f16 WGMMMA supports transposes”, FP16 decode ICE’d), file-cache stale-build bugs (need `CUTLASS_DSL_DISABLE_FILE_CACHING=1`), DSL closures not surviving nested `while` loops, region isolation forbidding `self.*` access in the kernel body.

References

- [1] Anonymous. Low-latency LLM inference via the megakernel, 2025. URL <https://neurips.cc/virtual/2025/loc/san-diego/poster/119124>. NeurIPS 2025 poster (San Diego).
- [2] DeepSeek-AI. Deepseek-v3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- [3] FlashInfer Project. FlashInfer: kernel library for LLM serving, including the TensorRT-LLM `trtllm_fp8_block_scale_moe` fused-MoE kernel, 2026. URL <https://github.com/flashinfer-ai/flashinfer>. Version benchmarked: `flashinfer.fused_moe.trtllm_fp8_block_scale_moe`, March 2026.
- [4] Gerard J. Holzmann. The model checker SPIN. *IEEE Transactions on Software Engineering*, 23(5): 279–295, 1997.
- [5] NVIDIA. CUTLASS python DSL (cute dsl), 2026. URL <https://github.com/NVIDIA/cutlass>.
- [6] Benjamin Spector, Jordan Juravsky, Stuart Sul, Owen Dugan, Dylan Lim, Dan Fu, Simran Arora, and Christopher Ré. Look ma, no bubbles! designing a low-latency megakernel for Llama-1B. Stanford Hazy Research blog, May 2025. URL <https://hazyresearch.stanford.edu/blog/2025-05-27-no-bubbles>.

A Per-iteration GPU timelines

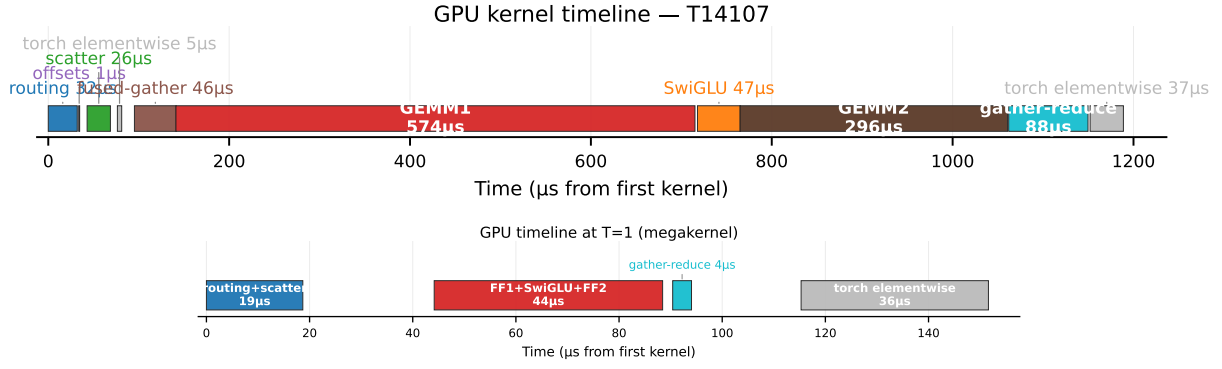


Figure 4: Per-iteration GPU timelines. **Top:** $T=14107$ — FF1 dominates ($574\mu s$); the routing kernel runs at $31\mu s$ thanks to the IEEE-twiddle Top-K (§2.1). **Bottom:** $T=1$ — the megakernel fuses routing+scatter ($19\mu s$) and FF1+SwiGLU+FF2 ($44\mu s$); total kernel latency is $56\mu s$, the trailing $36\mu s$ elementwise bar is a `do_bench` casting cost outside the kernel.